TOPIC 1 : INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward. Example 1: Input: words = ["abc","car","ada","racecar","cool"] Output: "ada" Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first. Example 2: Input: words = ["notapalindrome","racecar"] Output: "racecar" Explanation: The first and only string that is palindromic is "racecar".

Aim

To write a Python program to find the first palindromic string in a given array of strings. If no palindrome exists, return an empty string ("").

Algorithm

1. Start.
2. Define a function firstPalindrome(words).
3. Traverse each word in the list.
4. Check whether the word is equal to its reverse (word == word[::-1]).
5. If it is a palindrome, return the word immediately.
6. If no palindrome is found after checking all words, return an empty string ("").
7. Stop.

main.py Run

```
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

words = ["abc", "car", "ada", "racecar", "cool"]
print(firstPalindrome(words))
```

Output

```
ada
```

Time Complexity

O(n × m) where n is the number of words in the array and m is the average length of each string.

Space Complexity

O(1) auxiliary space as string reversal is checked in place without extra data structures.

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1. Return [answer1, answer2]. Example 1: Input: nums1 = [2,3,2], nums2 = [1,2] Output: [2,1] Example 2: Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6] Output: [3,4]

Aim

To write a Python program to count answer1 (number of elements in nums1 that exist in nums2) and answer2 (number of elements in nums2 that exist in nums1), returning [answer1, answer2].

Algorithm

1. Start.
2. Read the integer arrays nums1 and nums2.
3. Convert both arrays into sets set1 and set2 for O(1) lookup time.
4. Iterate through nums1 and count elements present in set2.
5. Iterate through nums2 and count elements present in set1.
6. Return the counts as a list [answer1, answer2].
7. Stop.

main.py Run

```python
def findIntersectionValues(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)
    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)
    return [answer1, answer2]

nums1 = [2, 3, 2]
nums2 = [1, 2]
print(findIntersectionValues(nums1, nums2))
```

Output

```
[2, 1]
```

Time Complexity

O(n + m) where n and m are the lengths of nums1 and nums2 respectively.

Space Complexity

O(n + m) to store unique elements of nums1 and nums2 in set data structures.

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as the number of distinct values in the subarray. Return the sum of the squares of distinct counts of all subarrays of nums. Example 1: Input: nums = [1,2,1] Output: 15 Explanation: Subarrays are [1] (1^2=1), [2] (1^2=1), [1] (1^2=1), [1,2] (2^2=4), [2,1] (2^2=4), [1,2,1] (2^2=4). Total = 1+1+1+4+4+4 = 15. Example 2: Input: nums = [1,1] Output: 3

Aim

To write a Python program to calculate the sum of squares of distinct counts of all possible contiguous subarrays of a given array.

Algorithm

1. Start.
2. Read the integer array nums.
3. Initialize total_sum = 0.
4. Outer loop i from 0 to len(nums) - 1 for starting index of subarray.
5. Inner loop j from i to len(nums) - 1 for ending index.
6. Maintain a set s to track distinct elements in nums[i..j].
7. Add (len(s))**2 to total_sum.
8. Return total_sum.
9. Stop.

main.py Run

```python
def sumCounts(nums):
    total = 0
    n = len(nums)
    for i in range(n):
        s = set()
        for j in range(i, n):
            s.add(nums[j])
            total += len(s) ** 2
    return total

nums = [1, 2, 1]
print(sumCounts(nums))
```

Output

```
15
```

Time Complexity

O(n^2) where n is the length of the array.

Space Complexity

O(n) for the set storing distinct elements in the current subarray.

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k. Example 1: Input: nums = [3,1,2,2,2,1,3], k = 2 Output: 4 Explanation: Pairs are (0,6), (2,3), (2,4), (3,4). Example 2: Input: nums = [1,2,3,4], k = 1 Output: 0

Aim

To write a Python program to count the number of valid index pairs (i, j) with identical values such that the product of their indices is divisible by k.

Algorithm

1. Start.
2. Read array nums and integer k.
3. Initialize count = 0.
4. Iterate i from 0 to n-1 and j from i+1 to n-1.
5. Check if nums[i] == nums[j] and (i * j) % k == 0.
6. If both conditions hold true, increment count by 1.
7. Print the final count.
8. Stop.

main.py Run

```python
def countPairs(nums, k):
    count = 0
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1
    return count

nums = [3, 1, 2, 2, 2, 1, 3]
k = 2
print(countPairs(nums, k))
```

Output

```
4
```

Time Complexity

$O(n^2)$ where n is the size of the array.

Space Complexity

O(1) constant auxiliary space.

5. Write a program FOR THE BELOW TEST CASES with least time complexity: Input: {1, 2, 3, 4, 5} Expected Output: 5 Input: {7, 7, 7, 7, 7} Expected Output: 7 Input: {-10, 2, 3, -4, 5} Expected Output: 5

Aim

To write a Python program to find the maximum element in an array with optimal linear time complexity O(n).

Algorithm

1. Start.
2. Read input array arr.
3. Assume largest = arr[0].
4. Iterate through each element num in arr.
5. If num > largest, update largest = num.
6. Return largest.
7. Stop.

main.py Run

```python
def find_largest(arr):
    if not arr:
        return None
    largest = arr[0]
    for num in arr:
        if num > largest:
            largest = num
    return largest

arr1 = [1, 2, 3, 4, 5]
arr2 = [7, 7, 7, 7, 7]
arr3 = [-10, 2, 3, -4, 5]
print(find_largest(arr1))
print(find_largest(arr2))
print(find_largest(arr3))
```

Output

```
5
7
5
```

Time Complexity

O(n) linear scan through the array.

Space Complexity

O(1) auxiliary memory.

6. Write a program that processes a list of numbers. It first sorts the list using an efficient sorting algorithm and then finds the maximum element in the sorted list. Test Cases: 1. Empty List: [] -> Output: None / List is empty 2. Single Element List: [5] -> Output: 5 3. All Elements Same: [3, 3, 3, 3, 3] -> Output: 3

Aim

To write a Python program to sort a list using TimSort (sort()) and extract the maximum element from the sorted array.

Algorithm

1. Start.
2. Check if list is empty. If empty, return 'List is empty'.
3. Sort the list in ascending order using sort().
4. The last element (arr[-1]) is the maximum element.
5. Return arr[-1].
6. Stop.

main.py Run

```python
def sort_and_find_max(nums):
    if not nums:
        return "List is empty"
    nums.sort()
    return nums[-1]

print(sort_and_find_max([]))
print(sort_and_find_max([5]))
print(sort_and_find_max([3, 3, 3, 3, 3]))
```

Output

```
List is empty
5
3
```

Time Complexity

O(n log n) for sorting the list using TimSort.

Space Complexity

O(1) or O(n) depending on in-place sorting.

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. Analyze its space complexity. Test Cases: Input: [3, 7, 3, 5, 2, 5, 9, 2] -> Output: [3, 7, 5, 2, 9] Input: [-1, 2, -1, 3, 2, -2] -> Output: [-1, 2, 3, -2] Input: [1000000, 999999, 1000000] -> Output: [1000000, 999999]

Aim

To write a Python program to construct a new list preserving order with only unique elements.

Algorithm

1. Start.
2. Initialize an empty list unique_list and a set seen.
3. Traverse each element in input list.
4. If element is not in seen, append to unique_list and add to seen.
5. Return unique_list.
6. Stop.

main.py Run

```
def get_unique_elements(nums):
    unique_list = []
    seen = set()
    for num in nums:
        if num not in seen:
            unique_list.append(num)
            seen.add(num)
    return unique_list

print(get_unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))
print(get_unique_elements([-1, 2, -1, 3, 2, -2]))
print(get_unique_elements([1000000, 999999, 1000000]))
```

Output

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

Time Complexity

O(n) time complexity with set lookup.

Space Complexity

O(n) auxiliary space for storing unique elements and seen set.

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code.

Aim

To write a Python program to implement Bubble Sort and analyze its Best, Average, and Worst case time complexity.

Algorithm

1. Start.
2. Pass through the array n-1 times.
3. Compare adjacent elements arr[j] and arr[j+1].
4. Swap them if arr[j] > arr[j+1].
5. If no swaps occur in a full pass, terminate early (optimized).
6. Return sorted array.
7. Stop.

main.py Run

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

arr = [64, 34, 25, 12, 22, 11, 90]
print("Sorted array:", bubble_sort(arr))
```

Output

```
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Time Complexity

Best Case: O(n) (already sorted), Average Case: $O(n^2)$, Worst Case: $O(n^2)$.

Space Complexity

O(1) in-place sorting space.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation. Test Case: Input X = {3, 4, 6, -9, 10, 8, 9, 30}, KEY = 10 -> Output: Element 10 is found at position 5 (after sorting) Input X = {3, 4, 6, -9, 10, 8, 9, 30}, KEY = 100 -> Output: Element 100 is not found

Aim

To write a Python program to search for a target element in a sorted array using Binary Search.

Algorithm

1. Start.
2. Sort input array arr.
3. Initialize low = 0, high = len(arr) - 1.
4. While low <= high:
5. mid = (low + high) // 2.
6. If arr[mid] == key, return position mid + 1.
7. If key < arr[mid], set high = mid - 1.
8. Else set low = mid + 1.
9. If element not found, return 'Element not found'.
10. Stop.

main.py Run

```python
def binary_search(arr, key):
    arr.sort()
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return f"Element {key} is found at position {mid + 1}"
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return f"Element {key} is not found"

arr = [3, 4, 6, -9, 10, 8, 9, 30]
print(binary_search(list(arr), 10))
print(binary_search(list(arr), 100))
```

Output

```
Element 10 is found at position 7
Element 100 is not found
```

Time Complexity

O(log n) for binary search on sorted array.

Space Complexity

O(1) auxiliary space.

10. Given an array of integers nums, sort the array in ascending order and return it without using built-in sorting functions in O(n log n) time complexity and minimum space complexity.

Aim

To write a Python program to perform Merge Sort using Divide and Conquer strategy.

Algorithm

1. Start.
2. If len(arr) <= 1, return arr.
3. Divide array into two halves at mid = len(arr) // 2.
4. Recursively apply merge_sort on left and right halves.
5. Merge two sorted halves using two-pointer traversal.
6. Return merged sorted array.
7. Stop.

main.py Run

```python
def mergeSort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        mergeSort(left)
        mergeSort(right)

        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    return arr

arr = [38, 27, 43, 3, 9, 82, 10]
print("Sorted Array:", mergeSort(arr))
```

Output

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
```

Time Complexity

O(n log n) in all cases (Best, Average, Worst).

Space Complexity

O(n) auxiliary space for temporary sub-arrays.